

Reproducing the Temeraire Huge-Page Aware Artifact Results on a Local Single Docker Setup

Aldo Costa

aldo.costa@hhu.de

Heinrich-Heine-Universität

Düsseldorf, Nordrhein-Westfalen, Deutschland

Abstract

Temeraire is a hugepage-aware extension of TCMalloc whose goal is to improve application performance by increasing hugepage coverage without inducing uncontrolled fragmentation. The original OSDI 2021 paper evaluates Temeraire through individual application studies, a Google fleet experiment, and a production rollout. This report focuses on the part of that evaluation that can be approached with public artifacts: the Redis 6.0.9 case study.

The report first provides the memory-management background needed to understand why hugepage-aware allocation matters, then summarizes the relevant aspects of Temeraire’s design. The reproduction artifact uses a Docker-based setup to compare a historical public TCMalloc revision in two configurations, one using the legacy pageheap path and one using the hugepage-aware path. The artifact records software versions, Transparent Huge Page settings, benchmark parameters, allocator preload state, and raw Redis benchmark output. The result is a deliberately narrow reproduction: it can test whether the Redis benchmark behavior is recoverable in a local setting, but it cannot reproduce Google’s production environment or validate the paper’s fleet-level claims.

CCS Concepts

• **Software and its engineering** → **Memory management**; • **Computer systems organization** → *Reliability*.

Keywords

memory allocation, huge pages, TCMalloc, Temeraire, Redis, reproducibility

ACM Reference Format:

Aldo Costa. 2026. Reproducing the Temeraire Huge-Page Aware Artifact Results on a Local Single Docker Setup. In *Proceedings of Master-Seminar*. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Memory allocation is often treated as a local implementation concern of C and C++ programs. In large systems, however, allocation decisions also govern where objects land in memory, whether huge pages can be exploited effectively, and how frequently the processor must perform costly address translation work. Hunter et al. make this point explicitly in the Temeraire paper: careful allocator design can improve “fleet efficiency” by reducing costs that manifest outside the allocator itself [1].

Temeraire modifies the pageheap of TCMalloc so that allocations are placed with awareness of hugepage-sized regions. The allocator is therefore not judged solely by the time spent inside `malloc` and `free`. Its placement decisions affect subsequent execution, because object layout influences hugepage coverage and hugepage coverage

influences TLB behavior. In some cases a more deliberate allocation decision is worthwhile even if the decision itself carries a small overhead.

A complete reproduction of the Temeraire paper is not feasible in a seminar setting. The paper’s strongest results come from Google’s production environment, where diverse workloads run across warehouse-scale infrastructure and large-scale telemetry is available to the authors. That setting is inaccessible to external researchers. The reproduction target here is therefore deliberately smaller: the Redis case study from the paper. Redis 6.0.9 is run under a legacy TCMalloc pageheap configuration and under a hugepage-aware configuration, with both variants measured across repeated `redis-benchmark` trials.

This report makes three contributions. It explains the memory-management background needed to understand why hugepage-aware allocation can affect application performance. It summarizes the relevant parts of Temeraire’s design. And it documents a public reproduction artifact with pinned source versions, a containerized setup, explicit benchmark parameters, preserved raw output, and documented deviations from the original experimental conditions.

2 Background

2.1 Paging and Address Translation

Modern operating systems give each process its own virtual address space, so program instructions operate on virtual rather than physical addresses. The processor and operating system translate virtual pages to physical page frames through page tables. Because walking these tables is expensive, processors cache recent translations in translation lookaside buffers (TLBs).

With ordinary 4 KiB pages, a large working set can require a large number of distinct translations. A 2 MiB memory region, for instance, decomposes into 512 ordinary pages; if the active working set exceeds the TLB’s effective reach, the processor must perform additional page-table walks that stall execution and reduce application throughput.

2.2 Huge Pages

Huge pages reduce this address-translation overhead by mapping larger virtual memory regions with fewer page-table entries. On x86 systems the standard huge page size is 2 MiB, so a single TLB entry covers the same range that would otherwise require 512 ordinary mappings. The benefit is not that 2 MiB of data are loaded into cache at once; it is that fewer cached translations are needed to cover the same amount of memory.

Realizing this benefit depends on memory layout. A hugepage mapping is useful only when the virtual range is suitably aligned

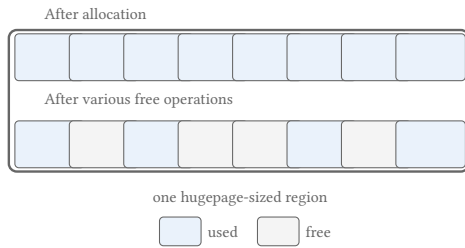


Figure 1: Allocation and deallocation patterns can create internal fragmentation. Free space exists within the hugepage-sized region, but live allocations prevent any contiguous range from being returned to the OS intact. Redrawn after Figure 1 in Hunter et al. [1].

and can be backed by a contiguous physical range. Linux Transparent Huge Pages (THP) can create such mappings automatically, but the kernel's ability to do so depends partly on the layout produced by the application allocator. When live objects are scattered across many hugepage-sized regions, the kernel has little opportunity to keep those regions as intact huge pages.

2.3 Fragmentation Trade-off

The difficulty lies in balancing memory efficiency against TLB efficiency. Returning unused memory in small pieces can reduce resident memory usage, but may force the kernel to split a hugepage mapping into ordinary pages. Retaining the hugepage intact preserves TLB coverage but keeps physical memory resident that the application no longer actively uses.

Figure 1 illustrates this in simplified form. The region still contains free memory, but the free space is interrupted by live allocations. Returning the free pieces to the OS risks losing hugepage coverage; retaining the whole region keeps physical memory partly idle. A hugepage-aware allocator must navigate this trade-off explicitly, seeking to keep hot memory on huge pages while allowing sufficiently empty regions to be released. Temeraire approaches this by changing the placement decisions made inside TCMalloc's pageheap.

3 Temeraire

3.1 TCMalloc Context

TCMalloc is organized as a hierarchy of allocation caches. Small allocations are typically served by per-CPU or per-thread caches before reaching a central cache; larger, page-granularity requests eventually reach the pageheap, which obtains memory from the OS and releases unused spans when possible. Temeraire modifies this backend layer rather than replacing the allocator as a whole.

Figure 2 shows the pageheap's position in the hierarchy. Many allocation requests are resolved in the upper cache layers and never reach the pageheap. This is architecturally convenient for Temeraire: because the pageheap is not on the critical path for every small allocation, it can afford more deliberate placement logic. The cost of that logic is expected to be recovered later, through better hugepage coverage and fewer TLB stalls in application code.

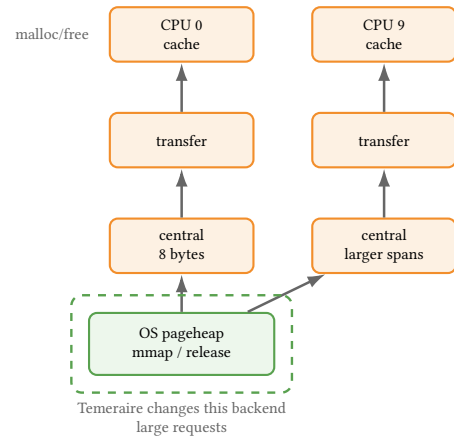


Figure 2: Simplified TCMalloc cache hierarchy. Temeraire replaces the pageheap backend without disturbing the upper cache layers. Redrawn after Figure 3 in Hunter et al. [1].

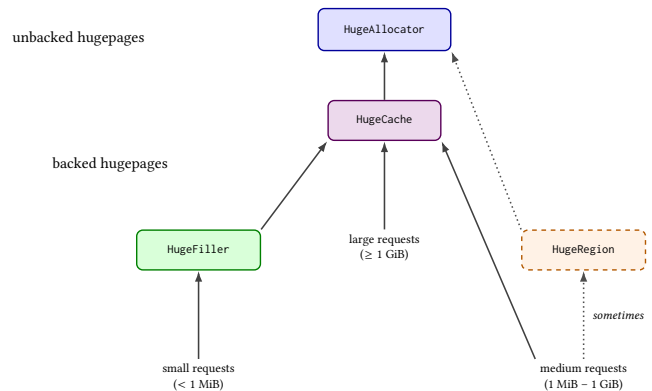


Figure 3: Temeraire's component structure. Small requests are routed to HugeFiller, large requests to HugeCache, and medium requests usually to HugeCache but occasionally to HugeRegion. HugeCache operates on backed hugepages; HugeAllocator manages unbacked hugepages. Adapted after Figure 6 in Hunter et al. [1].

3.2 Component Overview

Temeraire divides the pageheap backend into several cooperating components, shown in Figure 3.

The **HugeAllocator** manages hugepage-aligned virtual address ranges and supplies them to the other components on demand. The **HugeCache** retains fully empty, backed hugepages for reuse, avoiding the latency of immediately releasing and re-backing memory during short-term allocation fluctuations.

The **HugeFiller** handles small allocations. Rather than choosing any available hole, it applies placement policies designed to preserve useful free ranges and increase the probability that individual hugepages eventually become completely empty, at which point they can be safely released.

The `HugeRegion` addresses allocations too large for the filler but too small to round to a full hugepage without significant waste. Without this component, repeated allocations slightly above 2 MiB would consume nearly a full hugepage each. Regions allow such allocations to share a larger contiguous range and cross hugepage boundaries.

The top-level allocator routes requests between these components. Small requests go to the filler, while medium and large requests are directed to regions or full hugepage ranges depending on size. This size-based routing reflects that small, medium, and large allocations generate qualitatively different fragmentation risks.

3.3 Placement Heuristic

The central policy is implemented in `HugeFiller`. Ordinary best-fit allocation is insufficient because Temeraire cares not only about the total amount of free space within a hugepage, but also about its shape. A hugepage with one long contiguous free range is more valuable than one with many small holes, since it can accommodate larger future requests without fragmentation.

For a request of size K , Temeraire prefers hugepages whose longest free run is just sufficient for the request, thereby avoiding the unnecessary consumption of longer ranges. When multiple hugepages share the same longest-free-run status, the allocator prefers the fuller one, using allocation count as a proxy for proximity to becoming empty. The intuition is that a hugepage with few live allocations is more likely to drain naturally; disturbing it with a new allocation reduces that probability.

This heuristic captures the central design idea. Placement decisions are deliberately more conservative than a naive strategy because poor placement creates long-lived fragmentation, whereas careful placement preserves hugepage coverage and reduces subsequent TLB stalls. The pageheap's position outside the hot allocation path makes this additional reasoning affordable.

4 Related Work

Prior work on memory allocation and huge pages divides broadly into allocator-side and OS-side approaches. Temeraire belongs to the former category: it changes object placement in user space rather than relying on the kernel to recover hugepage-friendly layouts after the fact.

SuperMalloc is a scalable multithreaded allocator for 64-bit systems whose primary goals are allocation speed and scalability [3]. It uses huge pages for very large allocations, but this use is narrower than Temeraire's: SuperMalloc does not attempt to make the general pageheap layout hugepage-aware across a broad range of allocation sizes, nor does it address the fragmentation that arises when small and medium-sized allocations interact with THP.

MallocPool targets TLB efficiency through a different abstraction [2]. It preallocates a large memory region and serves chunks from it contiguously, allowing dynamically allocated objects to benefit from favorable TLB mappings. While related to Temeraire's focus on memory layout, MallocPool is not integrated into a general-purpose pageheap and does not need to balance the same combination of allocation sizes, release decisions, and production server workloads.

LLAMA uses learning-based lifetime prediction for C++ server workloads [5]. It co-locates allocations by predicted lifetime on

the grounds that huge pages are easier to release when all resident objects have compatible lifetimes. Mixing short-lived and long-lived objects on the same hugepage makes that page difficult to reclaim. Temeraire targets a related problem without lifetime prediction, relying instead on online placement heuristics, principally longest-free-range information and allocation count, that are available at allocation time without profiling.

On the OS side, Ingens and HawkEye manage huge pages from the kernel [4, 6]. These systems improve hugepage allocation, distribution, or reclamation without modifying the application allocator, making them complementary to Temeraire rather than competing with it. Kernel-level management is constrained by the virtual memory layout exposed from user space: if the allocator scatters live objects across hugepage-sized regions, the kernel faces fragmentation and memory pressure regardless of its own policies.

Taken together, these approaches illustrate why huge pages are attractive but difficult to exploit reliably. Temeraire's contribution is to make hugepage coverage an explicit objective inside a production allocator backend, packing some hugepages densely while allowing others to drain to the point where release is worthwhile.

5 Original Evaluation Methodology

The original paper evaluates Temeraire at several levels of granularity. Application-level results cover throughput, latency, memory usage, instructions per cycle, TLB-related stalls, and allocation overhead for individual production workloads. A Redis case study provides a more accessible comparison because Redis is open-source and its workload can be described precisely. Beyond these application experiments, the authors report a controlled fleet experiment and longitudinal data from a production rollout.

These levels answer distinct questions. The Redis case study asks whether the allocator change affects a concrete, publicly available application under controlled benchmark conditions. The fleet experiment and rollout ask whether hugepage-aware allocation improves aggregate efficiency across diverse warehouse-scale workloads. Only the former is amenable to external reproduction, and this artifact targets it exclusively.

The paper reports that the Redis experiment used Redis 6.0.9 and compared Temeraire with TCMalloc's legacy pageheap across repeated redis-benchmark trials. Each trial issued 1,000,000 requests combining a five-element push and a five-element read. These parameters are sufficiently explicit to form the basis of a local reproduction attempt.

6 Reproduction Scope

This artifact reproduces only the Redis case study. The fleet-scale results from the paper rely on Google's internal workload diversity, telemetry infrastructure, and production hardware, none of which are publicly accessible. The benchmark can therefore characterize how the Redis workload behaves on the tested machine under two allocator configurations; it cannot confirm the paper's aggregate efficiency claims, nor is it intended to.

7 Artifact Design

The repository is organized around a Docker container that provides a Linux environment with compiler tools, Redis tooling, perf, and

the required build dependencies. The host directory is mounted into the container so that raw outputs remain available in the repository tree after each run.

The setup script pins all relevant public source versions:

- Redis 6.0.9;
- gperftools-2.16, used only as an optional side baseline;
- public google/tcmalloc commit 8e534f507074.

The selected TCMalloc revision is a practical approximation. It still exposes the `want_no_hpa` mechanism needed to build a legacy-pageheap variant, but the public repository does not provide two ready-to-preload shared libraries for this experiment. Since the benchmark switches allocator modes through `LD_PRELOAD`, both configurations must be packaged as loadable shared objects.

To that end, the setup adds a small wrapper target inside the cloned TCMalloc tree. The wrapper contributes no allocator logic; it only provides the linkage needed for two shared libraries. One variant links against `//tcmalloc`. The legacy variant also links against `//tcmalloc:want_no_hpa`. The benchmark scripts then select `libtcmalloc_temeraire.so` or `libtcmalloc_legacy.so` at runtime through `LD_PRELOAD`.

This arrangement should not be read as a reconstruction of Google's internal allocator artifact. It is a public-code comparison that preserves the legacy-versus-hugepage-aware distinction at a level sufficient for a local Redis experiment.

8 Reproduction Methodology

The main workflow is:

```
docker compose build
docker compose run --rm temeraire-dev bash -lc \
  ". /scripts/setup_env.sh"
docker compose run --rm temeraire-dev bash -lc \
  ". /scripts/check_allocator_preload.sh"
docker compose run --rm temeraire-dev bash -lc \
  ". /scripts/collect_system_info.sh"
docker compose run --rm temeraire-dev bash -lc \
  ". /scripts/run_redis_benchmark.sh legacy"
docker compose run --rm temeraire-dev bash -lc \
  ". /scripts/run_redis_benchmark.sh temeraire"
```

For the final paper comparison, the repository also provides a paper-closer orchestration path:

```
docker compose run --rm temeraire-dev bash -lc \
  "echo always > /sys/kernel/mm/transparent_hugepage/enabled"
docker compose run --rm temeraire-dev bash -lc \
  "echo always > /sys/kernel/mm/transparent_hugepage/defrag"
docker compose run --rm temeraire-dev bash -lc \
  ". /scripts/run_paper_closer_redis_experiment.sh"
```

Repeated paper-closer runs can use a balanced allocator order:

```
docker compose run --rm temeraire-dev bash -lc \
  ". /scripts/run_paper_closer_redis_experiment.sh --
  allocator-order balanced"
```

This alternates which allocator is measured first across repeated runs and records the effective order in the manifest. It was added because the expected Redis effect is below one percent, so a fixed

allocator order can make host drift harder to separate from allocator behavior.

The benchmark script starts Redis, selects the allocator mode, records memory state, and executes repeated benchmark trials. Each trial flushes the database and then performs two operations:

- `LPUSH benchmark:list v1 v2 v3 v4 v5`;
- `LRANGE benchmark:list 0 4`.

The default configuration follows the paper-shaped Redis workload: 2000 trials, 1,000,000 requests per trial, 50 clients, and pipeline depth 16. The paper-closer script runs this workload in two modes, periodic release off and periodic release on, to match the two Redis rows in Table 1 of Hunter et al.: *Redis*[†] (periodic release disabled) and *Redis* (periodic release enabled). Results from the optional smoke-test mode, which reduces trial and request counts for environment validation, must not be compared against the paper.

Reproducibility metadata is collected prior to each benchmark run, covering kernel version, OS release, CPU model, memory summary, THP settings, Redis version, TCMalloc commit, and allocator artifacts. The allocator preload check is particularly important because Redis may report `mem_allocator: libc` even when `LD_PRELOAD` has substituted a different allocator at the process level.

The paper-closer rerun also confirmed that the LLVM toolchain path was exercised correctly: Redis and the allocator wrapper were compiled against a locally built LLVM 13.0.0 from commit `cd442157cfff4aad209ae532cbf031abbe10bc1df`. This closes one methodological gap relative to the paper, though differences in host hardware, hypervisor, and kernel behavior remain unavoidable.

A further complication arose during the initial paper-closer attempts. With the host in THP madvise mode under WSL2, process memory summaries showed no hugepage-backed anonymous memory throughout the entire run. The final rerun therefore set the shared WSL2/Docker THP policy to always before launching the benchmark, and added periodic `smaps_rollup` sampling so that hugepage activity could be observed mid-run rather than only in pre- and post-run snapshots.

The primary measured quantity is throughput in requests per second as reported by `redis-benchmark`. Raw benchmark logs are preserved in the artifact so that per-trial variance can be inspected directly. Where `perf` is available on the host, it can additionally collect hardware events including cycles, instructions, page faults, and TLB-related counters. These must be interpreted cautiously, however, given the effects of virtualization, container permissions, and counter multiplexing on reported values.

9 Results

The final analysis uses only runs that exercised the intended THP path. An earlier paper-closer attempt, performed while the WSL2 kernel exposed THP in madvise mode, recorded `AnonHugePages: 0 kB` for Redis and is therefore treated as workflow validation rather than as paper-comparable data. After switching the shared WSL2/Docker THP policy to always, every reported run showed non-zero `AnonHugePages` in Redis process snapshots.

For each Redis run, the artifact records separate LPUSH and LRANGE throughput. The comparison below uses a combined push-plus-read metric. For each trial it computes

$$2/(1/r_{\text{LPUSH}} + 1/r_{\text{LRANGE}})$$

and then averages across trials. This harmonic-mean formulation avoids treating the two operations as independent paper results, since Hunter et al. report only one Redis throughput delta per release mode.

Table 1: Run-level Temeraire throughput deltas in THP-enabled runs. L first and T first denote whether legacy or Temeraire ran first within the allocator pair. off/on denote periodic release disabled/enabled.

Run	Order	off	on
fixed-order	L first	+1.88%	+0.27%
balanced 1	L first	+0.44%	+0.12%
balanced 2	T first	+0.48%	+0.38%
balanced 3	L first	-0.76%	-2.37%
balanced 4	T first	+3.51%	-12.07%
targeted on	T first	-	+0.12%

Table 1 shows that release-off has the clearer local signal: four of five THP-enabled full runs favor Temeraire, and the median release-off delta is +0.48%. This is below, but close to, the paper’s +0.75% Redis[†] result. The release-on case is weaker. Most release-on runs are near flat or modestly positive, but one balanced run produced a large negative result.

That large negative release-on result was investigated separately. The Temeraire run from balanced rerun 4 had correct metadata, background release enabled, active THP, stable RSS, stable AnonHugePages, and a clean Redis log. Its slowdown was visible from the first 250-trial window and only partially recovered at the end of the run, while the paired legacy run was stable. A targeted release-on-only rerun with Temeraire first then returned to +0.12% combined throughput: LPUSH was -0.04% and LRANGE was +0.37%. This supports treating the -12.07% release-on result as a transient single-host outlier rather than as representative allocator behavior.

Table 2: Paper comparison and local interpretation.

Mode	Paper	Local summary	Interpretation
off	+0.75%	median +0.48%	recurring positive signal
on	+0.44%	about 0 to +0.38%	small and noisy

The result is therefore mixed but informative. The artifact does not reproduce the exact Redis Table 1 statistics. It does, however, repeatedly put the local Redis comparison in the paper’s small-effect regime once THP is actually active. Release-off is the stronger reproduction signal. Release-on remains close to the paper’s reported scale in most runs, but the single-host setup is noisy enough that the effect should be described as near-flat rather than robustly reproduced.

10 Discussion

The strongest aspect of this artifact is procedural reproducibility. It records exact public source versions, benchmark parameters, allocator mode, raw outputs, and system context, making the experiment inspectable and straightforward to rerun on different hardware. On the tested WSL2/i7-10700K host, the THP-enabled runs reproduced the underlying mechanism more convincingly than prior attempts: mid-run samples and end-state summaries confirm hugepage-backed anonymous memory. The balanced repetitions also make the result more scientifically useful than a single fixed-order run, because they expose the run-to-run variation that matters when the expected Redis effect is below one percent.

The principal limitation is environmental fidelity. The original Redis experiments were conducted on Intel Skylake Xeon servers using a specific LLVM revision and Google’s internal TCMalloc build environment. A Docker container on a consumer desktop cannot replicate the same kernel behavior, THP availability, physical memory fragmentation profile, NUMA topology, or hardware counter semantics. These factors are particularly consequential for hugepage experiments, since hugepage availability is a system-wide property that the benchmark process does not control.

Source identity is a related concern. The public TCMalloc commit used here is a historical approximation chosen because it still admits a clean legacy-versus-Temeraire split; it should not be taken as evidence that the internal artifact from the paper was reconstructed. The public TCMalloc repository has evolved substantially beyond the OSDI 2021 codebase, and the two have diverged in ways that are difficult to characterize from the outside.

Despite these limitations, the reproduction serves a concrete purpose. It translates the paper’s Redis methodology into explicit, runnable steps and provides a basis for discussing which claims in systems papers are verifiable from public artifacts and which depend on inaccessible production context. More concretely, it makes the paper’s central technical claim testable at small scale: allocator placement decisions can affect application performance through channels beyond the direct cost of malloc and free.

The main empirical lesson from the repeated runs is that local Redis results should be reported as a distribution, not as a single definitive number. Release-off repeatedly moves in Temeraire’s favor and is close to the paper’s reported scale. Release-on is more ambiguous: most runs are near flat or modestly positive, but the targeted rerun was needed to show that one very negative result was not reproducible. This is still useful evidence, but it is evidence about a noisy single-host approximation, not a confirmation of the paper’s exact production-era measurements.

11 Conclusion

This artifact provides a narrow local reproduction path for the Temeraire paper, deliberately scoped to the Redis case study. It does not attempt to reproduce the fleet-scale results, which depend on Google’s internal workload diversity and infrastructure. Its purpose is instead to make the underlying mechanism testable in a controlled, public setting.

The experiment should be read as local evidence about one public workload under two allocator configurations, not as a confirmation of the paper’s production claims. Under THP-enabled conditions,

release-off shows a recurring positive Temeraire signal in the same small-effect range as the paper. Release-on is smaller and noisier; after one severe outlier was rerun directly, the targeted result returned to a near-flat +0.12%. The final claim is therefore intentionally limited: the artifact reproduces the Redis methodology and often lands in the paper's effect regime, but it does not robustly reproduce the exact Redis Table 1 statistics.

Its broader contribution is to make the mechanism concrete: allocation affects performance not only through the speed of `malloc` and `free`, but also through the memory layout it produces. Huge pages can reduce TLB pressure, but they make fragmentation more consequential. Temeraire addresses this by packing allocations so that some hugepages remain dense while others can drain to the point where release is worthwhile, a design goal that even a local Docker experiment can begin to probe.

References

- [1] A.H. Hunter, Chris Kennelly, Paul Turner, Darryl Gove, Tipp Moseley, and Parthasarathy Ranganathan. 2021. Beyond malloc efficiency to fleet efficiency: a hugepage-aware memory allocator. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*. USENIX Association, 257–273. <https://www.usenix.org/conference/osdi21/presentation/hunter>
- [2] Marcus Jägemar. 2018. Mallocpool: Improving Memory Performance Through Contiguously TLB Mapped Memory. In *2018 IEEE 23rd International Conference on Emerging Technologies and Factory Automation (ETFA)*, Vol. 1. 1127–1130. doi:10.1109/ETFA.2018.8502619
- [3] Bradley C. Kuszmaul. 2015. SuperMalloc: a super fast multithreaded malloc for 64-bit machines. In *Proceedings of the 2015 International Symposium on Memory Management (Portland, OR, USA) (ISMM '15)*. Association for Computing Machinery, New York, NY, USA, 41–55. doi:10.1145/2754169.2754178
- [4] Youngjin Kwon, Hangchen Yu, Simon Peter, Christopher J. Rossbach, and Emmett Witchel. 2016. Coordinated and efficient huge page management with ingens. In *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation (Savannah, GA, USA) (OSDI'16)*. USENIX Association, USA, 705–721.
- [5] Martin Maas, David G. Andersen, Michael Isard, Mohammad Mahdi Javanmard, Kathryn S. McKinley, and Colin Raffel. 2020. Learning-based Memory Allocation for C++ Server Workloads. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (Lausanne, Switzerland) (ASPLOS '20)*. Association for Computing Machinery, New York, NY, USA, 541–556. doi:10.1145/3373376.3378525
- [6] Ashish Panwar, Sorav Bansal, and K. Gopinath. 2019. HawkEye: Efficient Fine-grained OS Support for Huge Pages. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (Providence, RI, USA) (ASPLOS '19)*. Association for Computing Machinery, New York, NY, USA, 347–360. doi:10.1145/3297858.3304064